SpacerScope User Manual

1. Introduction

What is SpacerScope?

SpacerScope is a high-efficiency off-target analysis tool designed for **CRISPR/Cas genome editing** applications. It is intended to address the limitations of traditional methods on large reference genomes, especially complex plant genomes, where computational cost is high, response is slow, and it is difficult to balance speed and accuracy.

The core idea of SpacerScope is to divide off-target searching into three stages: **site extraction, fast prefiltering, and exact validation**. The program first extracts candidate spacer sites that satisfy PAM constraints from both the query sequence and the reference genome. It then uses different optimized encoding strategies for different search tasks: in **mismatch search**, it uses compact **2-bit sequence encoding** and batch scanning for rapid Hamming-distance evaluation; in **insertion/deletion search**, it first encodes sequences into multi-channel binary features, then uses **channel filtering and batch mask fusion** to greatly compress the candidate set before performing exact alignment on the retained small subset. For indel validation, SpacerScope uses **right-end-anchored edit-distance dynamic programming**, rather than simple local alignment, making it better suited to CRISPR spacer scenarios where insertions, deletions, and terminal constraints coexist.

Therefore, SpacerScope does not rely solely on conventional full-sequence alignment. Instead, it combines **efficient binary-encoding-driven prefiltering** with **exact edit-distance validation**, significantly reducing overall search cost while preserving result completeness and accuracy. Practical testing shows that SpacerScope can provide fast, stable, and high-precision gRNA screening and off-target analysis in complex genomic backgrounds.

SpacerScope Core Features

1. **Comprehensive off-target detection capability**
   Under user-defined constraints such as PAM, mismatch count, and indel count, SpacerScope performs systematic searches over eligible candidate sites and aims to preserve result completeness and decision accuracy during exact validation.

2. **Support for both mismatch and indel off-target analysis**
   SpacerScope supports not only standard **base substitution (mismatch)** detection, but also **insertion** and **deletion** off-target identification, covering both common and more complex potential CRISPR/Cas off-target types.
3. **Efficient binary encoding and prefiltering mechanisms**
   The program uses task-specific binary representations and prefilter strategies:
   - In mismatch search, it uses compact **2-bit encoding** and batch scanning to accelerate large-scale Hamming-distance computation.
   - In indel search, it uses multi-channel encoding, binary filtering, and batch mask fusion to significantly reduce the number of candidates entering exact alignment.
     This "fast filtering first, exact validation later" design substantially improves overall runtime efficiency.
4. **Suitable for large-scale genome analysis**
   Through optimizations in memory access, parallel scheduling, and candidate filtering, SpacerScope maintains good computational efficiency on large reference genomes, especially in high-load scenarios such as complex plant genomes.
5. **Multiple input modes and analysis workflows**
   SpacerScope supports:
   - direct FASTA query input;
   - analysis based on target sequences extracted from annotation files;
   - both standard mode and low-memory cut mode.
     In cut mode, the reference genome is processed chromosome by chromosome in a streaming fashion on machines with limited memory resources.
6. **Cross-platform compatibility**
   SpacerScope provides support for **Windows**, **Linux**, and **Mac**, and can be used either as a command-line tool or through `spacerscope-web` in a browser. Because of the specifics of the Mac platform, users need to compile and package it themselves.
7. **Parallel execution and low-level optimization support**
   The program supports multithreaded execution and, on supported hardware and compiler environments, can take advantage of instruction-set optimizations such as **AVX2** and **SSE4.1** to further improve practical performance.
8. **Adaptable to multiple CRISPR systems**
   Users can define custom **PAM sequences** and their relative orientation, allowing adaptation to different nuclease systems and experimental design requirements.

---

2. Program Components

Current releases typically include two executable programs:

- `spacerscope`: the main program
- `spacerscope-web`: the web GUI shell, which internally invokes `spacerscope` as a subprocess

Their relationship is as follows:

- `spacerscope` performs all actual computation and can be run independently.

- `spacerscope-web` provides a browser-based GUI and also facilitates network-based access. It is not an independent search engine and must be used together with `spacerscope`.

---

3. Environment Requirements

Supported Operating Systems

- Windows x64
- Linux x86_64
- Mac

CPU Instruction Sets

Recommended CPU support:

- AVX2
- SSE4.1

On Linux, you may check support using:

```
lscpu | grep -E 'avx2|sse4_1' | sed 's/^[[:space:]]*//'
```

The program can still run on machines without AVX2 support, but performance may be reduced.

Runtime Dependencies

Core dependencies:

- OpenMP
- zlib

If you use `spacerscope-web`, no additional software is required beyond a browser; you only need to make sure the `spacerscope` main program can be found in the same directory or at the specified path.

---

4. Running Modes

SpacerScope currently supports four main modes:

Mode	Input type	Memory strategy	Description
`fasta`	Query FASTA + reference FASTA	Standard mode	Directly use FASTA query sequences
`fasta-cut`	Query FASTA + reference FASTA	Low-memory mode	Stream the reference sequence chromosome by chromosome

Mode	Input type	Memory strategy	Description
`anno`	Annotation file + gene ID + reference FASTA	Standard mode	Extract query sequences from annotation, then run
`anno-cut`	Annotation file + gene ID + reference FASTA	Low-memory mode	Annotation extraction plus chromosome-wise low-memory processing

Difference Between Standard Mode and Cut Mode

- **Standard mode**: the entire reference genome is loaded into memory, which is usually faster.
- **Cut mode**: the reference genome is processed one chromosome at a time, which reduces peak memory and is more suitable for memory-limited machines.

Peak Memory Estimation

At startup, the program automatically estimates peak memory usage based on the reference FASTA and asks for user confirmation. To skip this step, use:

```
-y
```

or

```
--yes
```

or

```
--skip-memory-check
```

- In standard mode, peak usage mainly depends on the **total genome size**.
- In cut mode, peak usage mainly depends on the **size of the longest chromosome**.

---

5. Command-Line Program: `spacerscope`

General Usage

```
spacerscope fasta <options>
spacerscope fasta-cut <options>
spacerscope anno <options>
spacerscope anno-cut <options>
spacerscope --report-tsv <file> --html-output <file>
```

Show help:

```
spacerscope --help
spacerscope help fasta
spacerscope help anno
```

Show version:

```
spacerscope --version
```

Common Parameters

Parameter	Description	Default
`-R, --ref <file>`	Reference genome FASTA file	Required
`-o, --output <file>`	Output TSV file	Required
`--pam <string>`	Primary PAM sequence	`NGG`
`--alt-pam <string>`	Secondary PAM sequence	`NAG`
`--spacer-len <int>`	Spacer length	`20`
`--direction <up|down>`	Spacer orientation relative to PAM	`up`
`-m, --mismatch <int>`	Maximum number of mismatches	`4`
`-i, --indel <int>`	Maximum number of indels	`2`
`--dust-threshold <int>`	DUST low-complexity filtering threshold	`12`
`-t, --threads <int>`	Number of threads	Logical core count
`-y, --yes`	Skip pre-run memory confirmation	Off
`--skip-memory-check`	Skip pre-run memory confirmation	Off
`--raw-output`	Write a 7-column unscored TSV	Off
`--progress-format <text|jsonl>`	Progress output format	`text`
`--html <0|1>`	Whether to generate `<output>.html`	`1`
`-ht <0|1>`	Alias of `--html`	`1`
`--html-output <file>`	Specify the HTML output file	Empty
`--report-tsv <file>`	Generate HTML from an existing scored TSV only	Empty

Parameters for `fasta / fasta-cut` Mode

Parameter	Description
`-I, --query <file>`	Query FASTA file

Examples

```
spacerscope fasta --query q.fa --ref ref.fa -o out.tsv
```

```
spacerscope fasta-cut --query q.fa --ref ref.fa -o out.tsv --yes
```

Parameters for `anno / anno-cut` Mode

Parameter	Description
`-A, --annotation <file>`	Annotation file (GFF/GFF3)
`--geneID <id>`	**Exact gene ID** in the annotation file
`--seq-type <gene|mrna|cds>`	Sequence level to extract; default is `gene`

Examples

```
spacerscope anno --annotation genes.gff3 --geneID gene:Gene1 --seq-type cds --ref ref.fa
-o out.tsv
```

```
spacerscope anno-cut --annotation genes.gff3 --geneID gene:Gene1 --seq-type gene --ref
ref.fa -o out.tsv --yes
```

Important Limitations of `anno` Mode

In the current version, `anno / anno-cut` has been merged into the C++ main program, but its annotation compatibility strategy is **released under legacy conventions**, not as a universal parser for arbitrary GFF3 files.

Please note:

1. `--geneID` must match the **gene ID in the annotation file exactly**; it is not a free-form gene symbol.
2. `mrna/cds` extraction follows legacy conventions: transcript association depends on the `GeneX-mRNA` **naming pattern**.
3. For generic GFF3 files using styles such as `Parent=transcript:...`, the current version **does not guarantee** full compatibility in `mrna/cds` mode.

If your annotation file is complex or inconsistent, it is recommended to:

- manually extract the target gene sequence first;
- then run SpacerScope in `fasta` or `fasta-cut` mode.

`--raw-output` and HTML Reports

In the current version:

- Normal output: **9-column scored TSV**
- `--raw-output`: **7-column unscored merged TSV**

The file generated by `--raw-output` contains:

Column name
QueryName
QuerySeq
TargetName
TargetSeq
Distance
SearchType
GenomicFrequency

Notes:

- `--raw-output` and `--html-output` **cannot be used together**
- HTML reports can only be generated from scored TSV output

Generate HTML Only from an Existing TSV

If you already have a scored TSV, you can generate HTML separately:

```
spacerscope --report-tsv out.tsv --html-output out.html
```

---

6. Web Version (GUI): `spacerscope-web`

Suitable Scenarios

`spacerscope-web` is the local web shell provided by the current version. It is suitable for:

- Linux servers without a desktop environment
- users who prefer configuring parameters and viewing results in a browser
- accessing the local service through SSH port forwarding

How to Start

```
spacerscope-web
```

(Requirement: the `spacerscope` main program must be present in the same path.)

Default listening address:

```
127.0.0.1:8787
```

Common parameters:

Parameter	Description	Default
`--bind <host>`	Bind address	`127.0.0.1`
`--port <int>`	Listening port	`8787`
`--workspace-root <dir>`	Web working directory	`spacerscope-web` under the system temp directory
`--spacerscope-bin <path>`	Path to the `spacerscope` main binary	Auto-discovered in the same directory
`-h, --help`	Show help	-
`-V, --version`	Show version	-

Examples

```
spacerscope-web --bind 127.0.0.1 --port 8787
spacerscope-web
```

Access

Open in a browser:

```
http://127.0.0.1:8787
```

If you use it on a remote Linux server, SSH port forwarding is recommended:

```
ssh -L 8787:127.0.0.1:8787 user@server
```

Then open locally in your browser:

```
http://127.0.0.1:8787
```

Web Version Notes

The web page supports:

- `fasta / fasta-cut / anno / anno-cut`
- server-path input mode
- file upload mode
- Chinese/English switching
- real-time logs
- real-time progress
- TSV / HTML / stdout / stderr download

6.5 Security Notice

Current `spacerscope-web` v1 **has no authentication mechanism**.

Therefore:

- it should be bound only to `127.0.0.1` by default
- if bound to `0.0.0.0` or another LAN address, it should only be used in a **trusted network**
- direct exposure to the public internet is not recommended

---

7. Output Files

Default Main Output (scored TSV)

The default output is a **9-column scored TSV**:

Column name	Meaning
`QueryName`	Query site name
`QuerySeq`	Query site sequence
`TargetName`	Off-target site name
`TargetSeq`	Off-target site sequence
`Distance`	Edit distance
`SearchType`	Match type: `Exact` / `Mismatch` / `Indel`
`GenomicFrequency`	Number of occurrences of this sequence in the genome
`Score`	Off-target score
`Details`	Alignment details

HTML Report

If HTML output is enabled, the program additionally generates:

```
<output>.html
```

or the file you specify with `--html-output`.

The HTML report includes:

- summary statistics for each query
- distribution of exact / mismatch / indel hits
- detailed result tables

Raw TSV

When `--raw-output` is used, the program writes a **7-column unscored TSV**, without:

- `Score`
- `Details`

At the same time, no HTML report will be generated.

---

8. Parameter and Runtime Recommendations

General Recommendations

- In most cases, the default parameters are already usable.
- Unless there is a clear need, it is not recommended to arbitrarily increase `--indel`.
- It is best to start with the default parameters and then fine-tune as needed.

About `--indel`

Increasing `--indel` will noticeably increase computational cost.

Typical recommendations:

- `--indel 0`: exact matches plus mismatch search only
- `--indel 1~2`: the usual recommended range
- Higher `--indel`: try only when clearly necessary

About `--threads`

The program supports multithreading, but more threads are not always better. Actual speed depends on factors such as memory bandwidth, NUMA, and machine architecture.

Recommended practice:

- first test with the default thread count or a commonly used high-performance thread count on the machine

- determine the best value empirically on servers

8.4 About Cut Mode

The purpose of cut mode is to **reduce peak memory usage**. It is not always faster. When memory is sufficient, standard mode is usually more straightforward; when memory is limited, cut mode is safer.

---

9. Notes and Known Limitations

1. It is recommended to place all relevant files under **English-only paths** when using the program.
2. It is not recommended to run the entire genome directly as the query FASTA.
3. It is recommended to use **complete contiguous gene sequences** as query input files (for example, in `fasta` mode). Directly using mRNA or CDS-level sequences as queries may lead to designed gRNA sequences that do not exist in the genome because of splicing, resulting in zero exact matches or exact matches not located on the target gene.
4. For annotation files:
   - the current `anno` mode is not a universal parser for arbitrary GFF3
   - `mrna/cds` mode especially depends on legacy annotation conventions
5. Because annotation file formats are not uniform across datasets, it is recommended to extract the **gene sequence** first and use **fasta mode**.
6. `spacerscope-web` v1 has no authentication and is not recommended for direct public exposure.
7. `--progress-format jsonl` is mainly intended for the web shell and programmatic invocation; most users should simply use the default `text` format.
8. When choosing the final gRNA, make sure that all exact-match sites are located on the **target gene**.
9. Make sure the software is granted sufficient permissions during use.

---

10. Quick Command Reference

FASTA Mode

```
spacerscope fasta --query query.fa --ref genome.fa -o result.tsv
```

FASTA-CUT Mode

```
spacerscope fasta-cut --query query.fa --ref genome.fa -o result.tsv --yes
```

ANNO Mode

```
spacerscope anno --annotation genes.gff3 --geneID gene:Gene1 --seq-type gene --ref genome.fa -o result.tsv
```

ANNO-CUT Mode

```
spacerscope anno-cut --annotation genes.gff3 --geneID gene:Gene1 --seq-type cds --ref genome.fa -o result.tsv --yes
```

Generate HTML Only

```
spacerscope --report-tsv result.tsv --html-output result.html
```

Start the Web Shell

```
spacerscope-web --bind 127.0.0.1 --port 8787
```